

Discovery of the **Cloud Native applications** from an **application security** perspective.



Dominique Righetto
Intrusion & Application Security team
Excellium Luxembourg entity

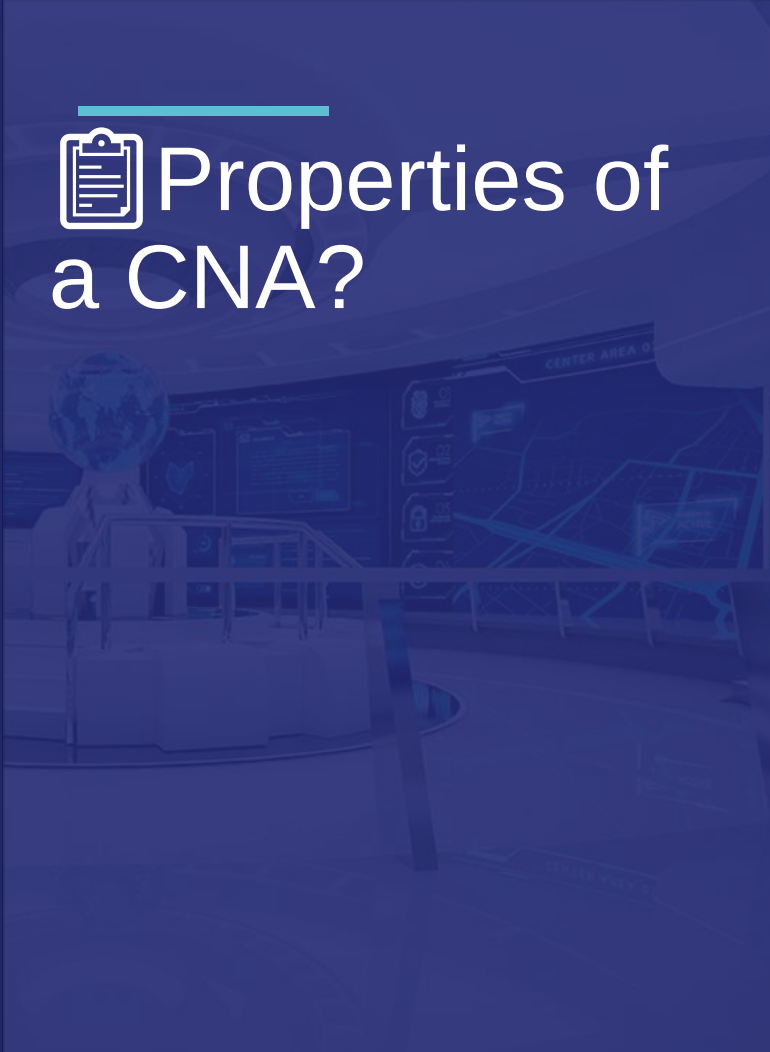


Convention:
Cloud Native Application will be called **CNA**.



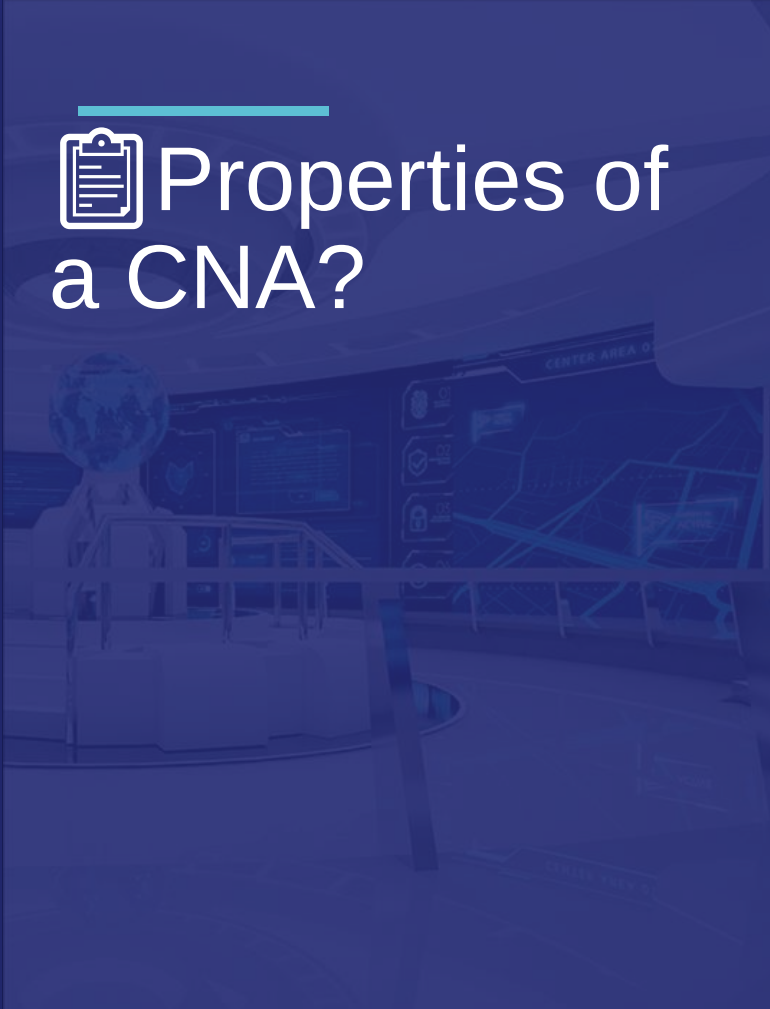
Question:

Which aspects of the security of an application will change, when an application will be intended to be a Cloud Native one?



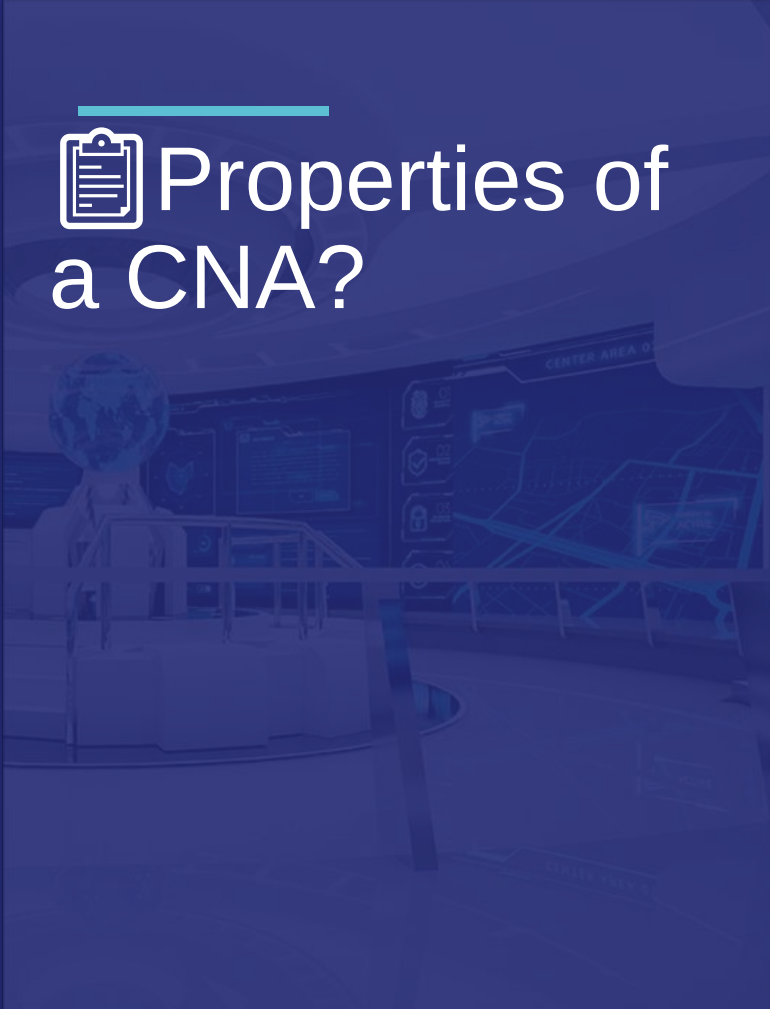
Properties of a CNA?

- Leverage a **maximum of services provided** by a Cloud provider to **focus on the added value aspect** of the application.
- It is a distributed system by design.
- It is **stateless** to facilitate easy horizontal scaling in and out.



Properties of a CNA?

- It is composed of **several dedicated parts**:
 - Each one leveraging **the more effective Cloud feature** according to his business/technical objective.
 - A **part** can be a serverless function or a collection of microservices hosted in a container managed by a container orchestrator.



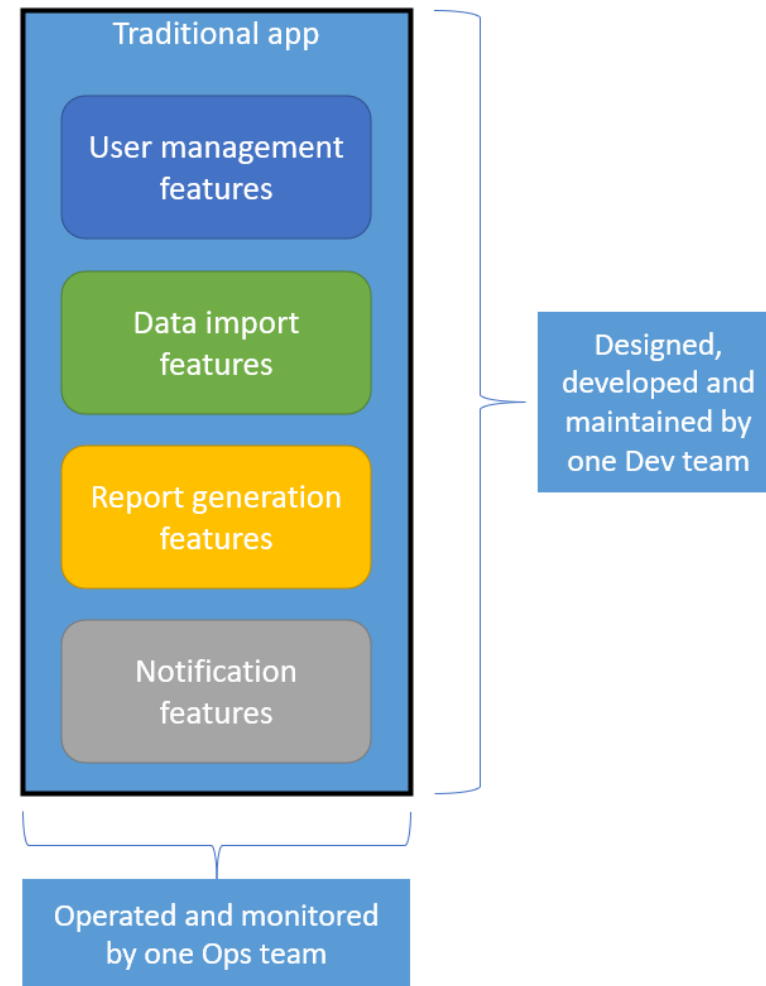
Properties of a CNA?

- Its **design and implementation anticipated** a kind of issues that can occur in Cloud-based environment.

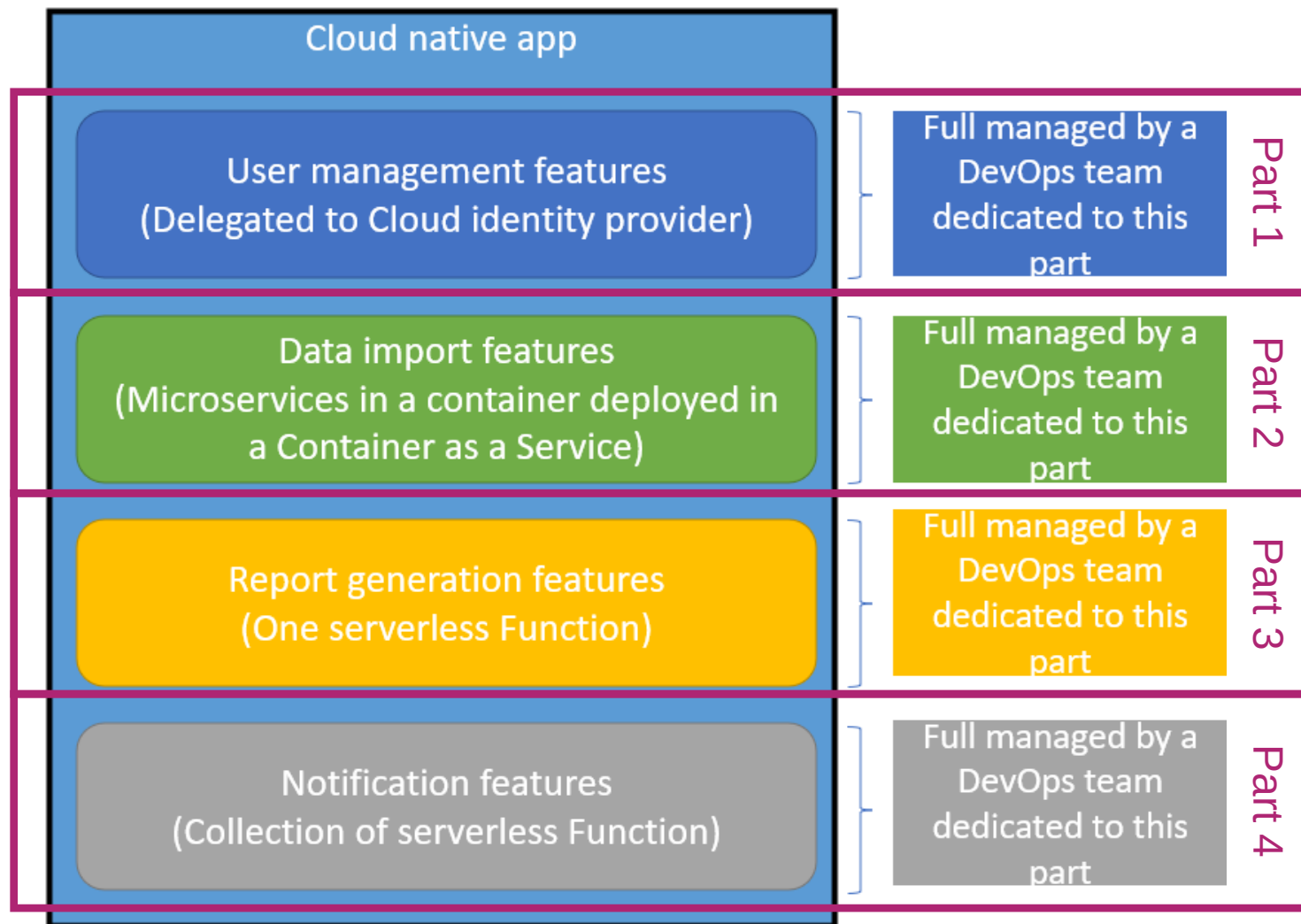
 Example:

Short network disruption, so they have built-in “retry” mechanisms.

Properties of a CNA?



Properties of a CNA?



Challenges brought by a CNA?

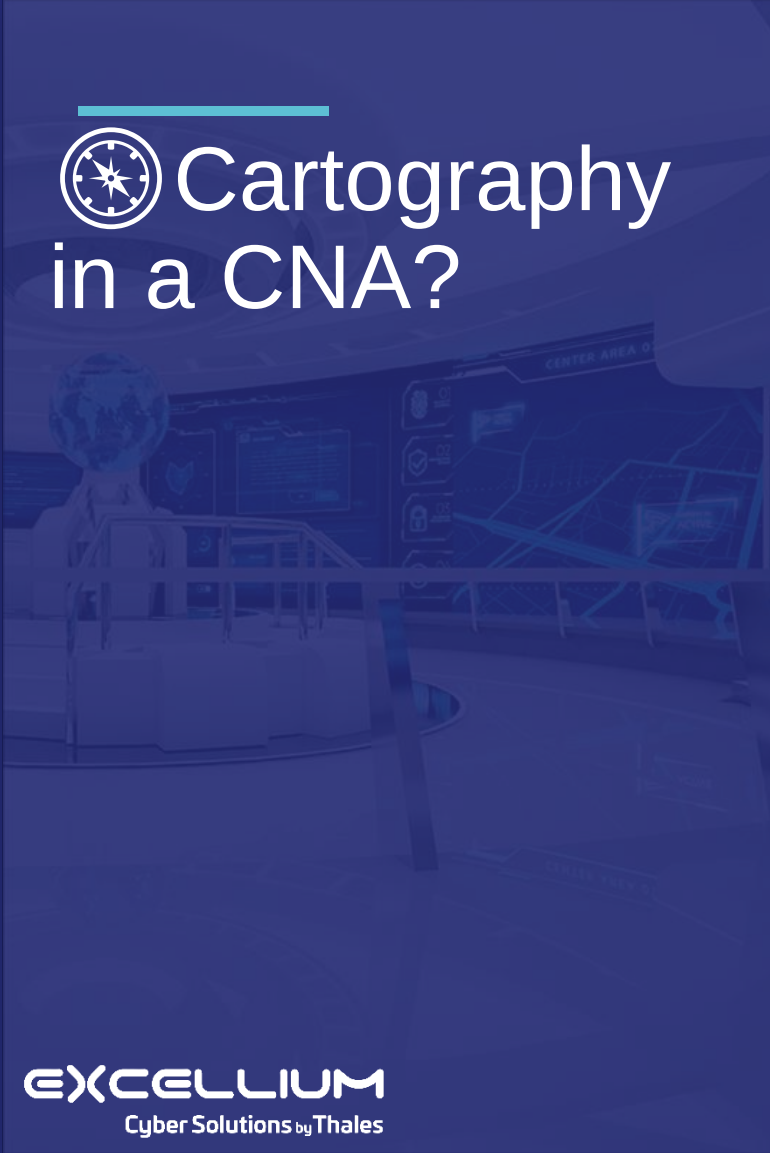
- Global security posture of the CNA is defined **by the sum of the security posture of each part.**
- Each part must has the same security level.
- Security cannot be handled **anymore by a single app layer** (ex: service layer).

🔑 Challenges brought by a CNA?

- Logging must be **unified across all parts** in terms of information used and format.
- User triggered events **need to be correlated across all parts**.

🔑 Challenges brought by a CNA?

- Each part will be developed and operated **by a separated DevOps team**.
- Each DevOps team has its own:
 - Maturity and knowledge in terms of application security.
 - Development velocity and timeline.
 - Development methodology and process.
 - Technology stack.

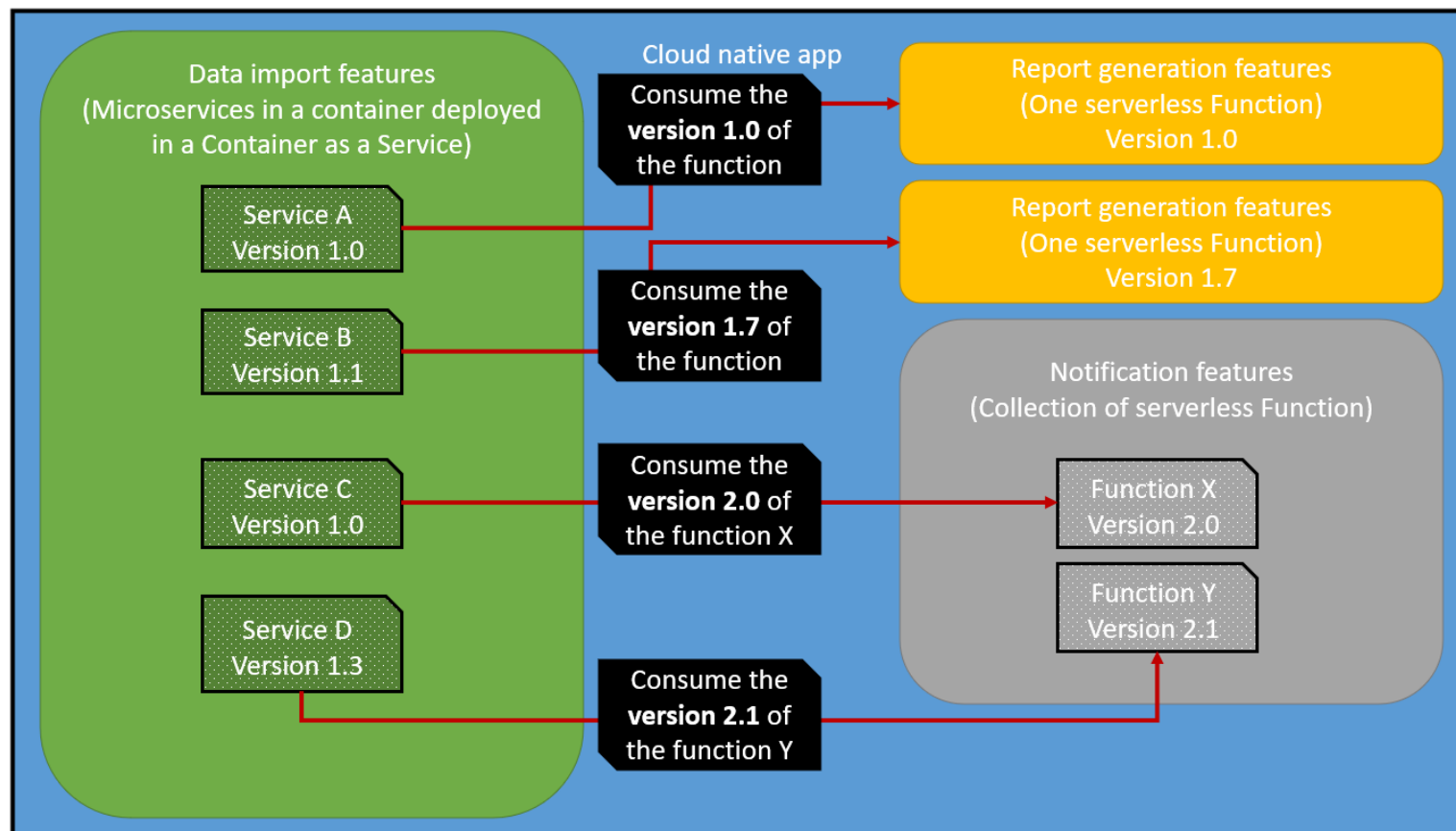


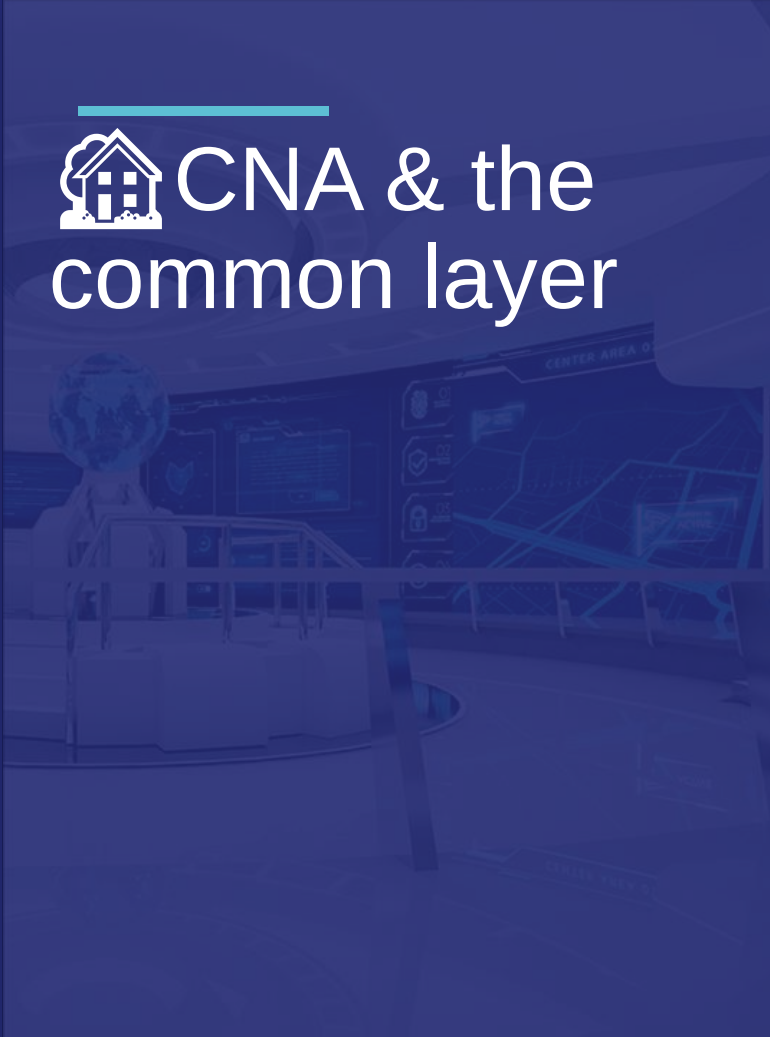
Cartography in a CNA?

EXCELLIUM
Cyber Solutions by Thales

- Each part will evolve in a **different timeline or velocity**.
- It requires to manage **different versions of a part** to ensure a **correct running of the whole CNA**.

Cartography in a CNA?



A graphic on the left side of the slide. It features a dark blue background with a faint, stylized illustration of a control room or mission center. In the foreground, there's a desk with a computer monitor displaying various data and maps. In the background, there are more screens and a globe. The text 'CNA & the common layer' is written in white, with 'CNA' in a larger font. Above the text is a small icon of a house with a cloud-like shape above it. The overall aesthetic is technical and modern.

CNA & the common layer

- A **common property of all parts** of a CNA is the following:
 - Almost all **microservices** will run **in a container managed by a container orchestrator**.
 - Almost all **serverless functions** will run in a context in which **processing can be applied to in/out flow**.



CNA & the common layer

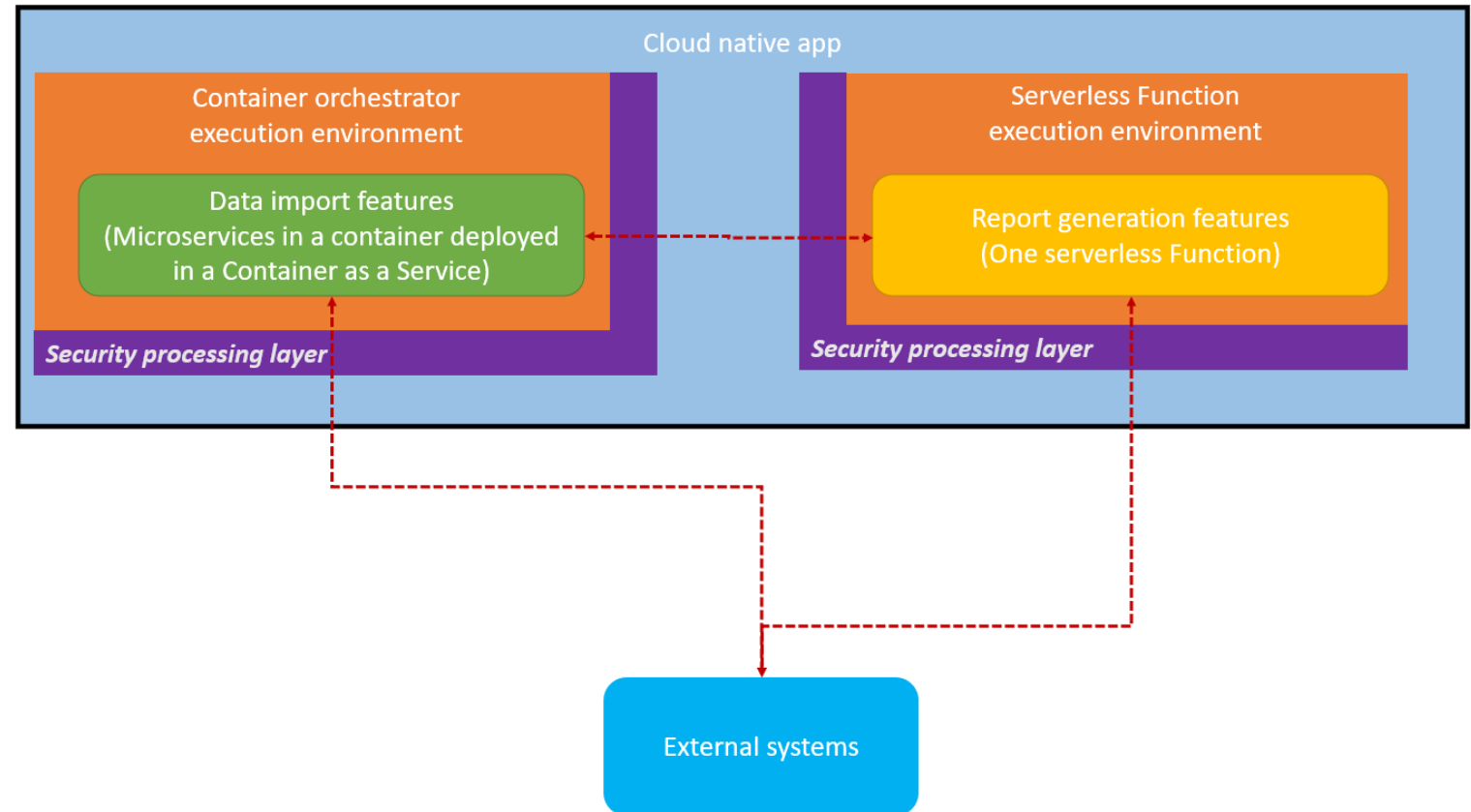
- 💡 Idea n°1: Leverage the **execution environment**, when it is possible.
- 💡 Idea n°2: Add security aspects **at execution environment level** instead to ask to a DevOps team to add them **at part level itself**.
- 💡 Idea n°3: Leverage a maximum of **built-in security features** provided by the web framework used.



CNA & the common layer

- DevOps team can **continue to focus on the business purpose** of its part of the CNA.
- DevOps team is not (**or the most minimally possible**) disturbed with security-related additional works and tasks.

CNA & the common layer



CNA & the common layer

 Case of the **containerized microservices managed by an orchestrator** was taken here because it is available as “*on-prem*” or “*cloud-service*” mode.

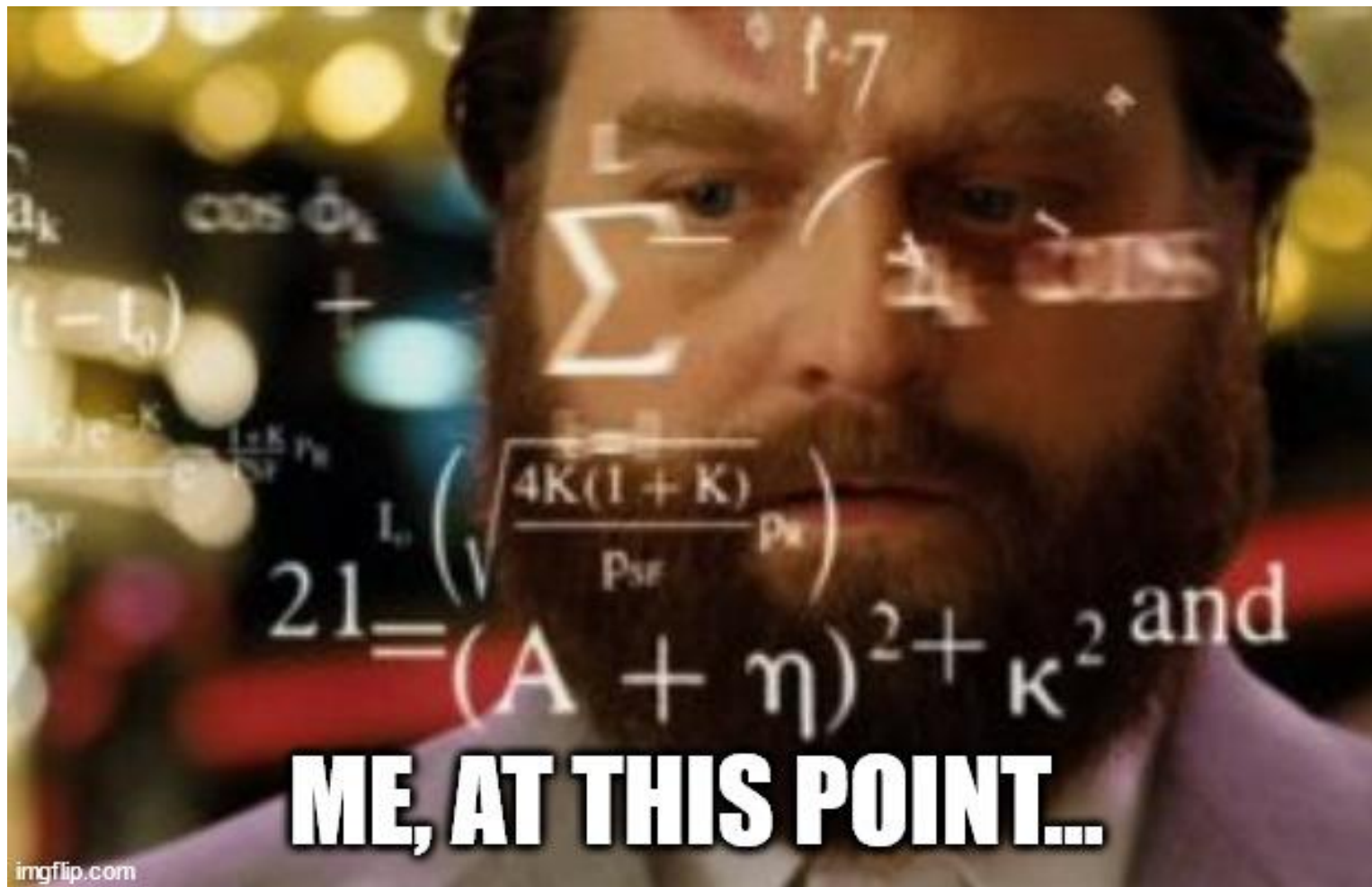
 For **serverless** case:
The security related processing can be added via the “*extensions*” and “*configuration*” features.

Legend:

-  Container orchestrator level.
-  Service mesh level.
-  Application level (code or CI/CD pipeline).

Area	Handled at which level?
Authentication.	
Authorization.	 When authorization is at service level only  +  When authorization is at data level
Input data validation and output data encoding.	
Error handling and technical information disclosure prevention.	 For app part  For service mesh related part
Protection of data in transit and at rest.	 For data at rest  For data in transit
Secure credentials/secret handling and storage.	
Security logging and monitoring.	
Monitoring of third-party components used from a security perspective to prevent to use vulnerable one.	 In CI pipeline

 Source for the area: [OWASP Top 10 Proactive Controls](#)



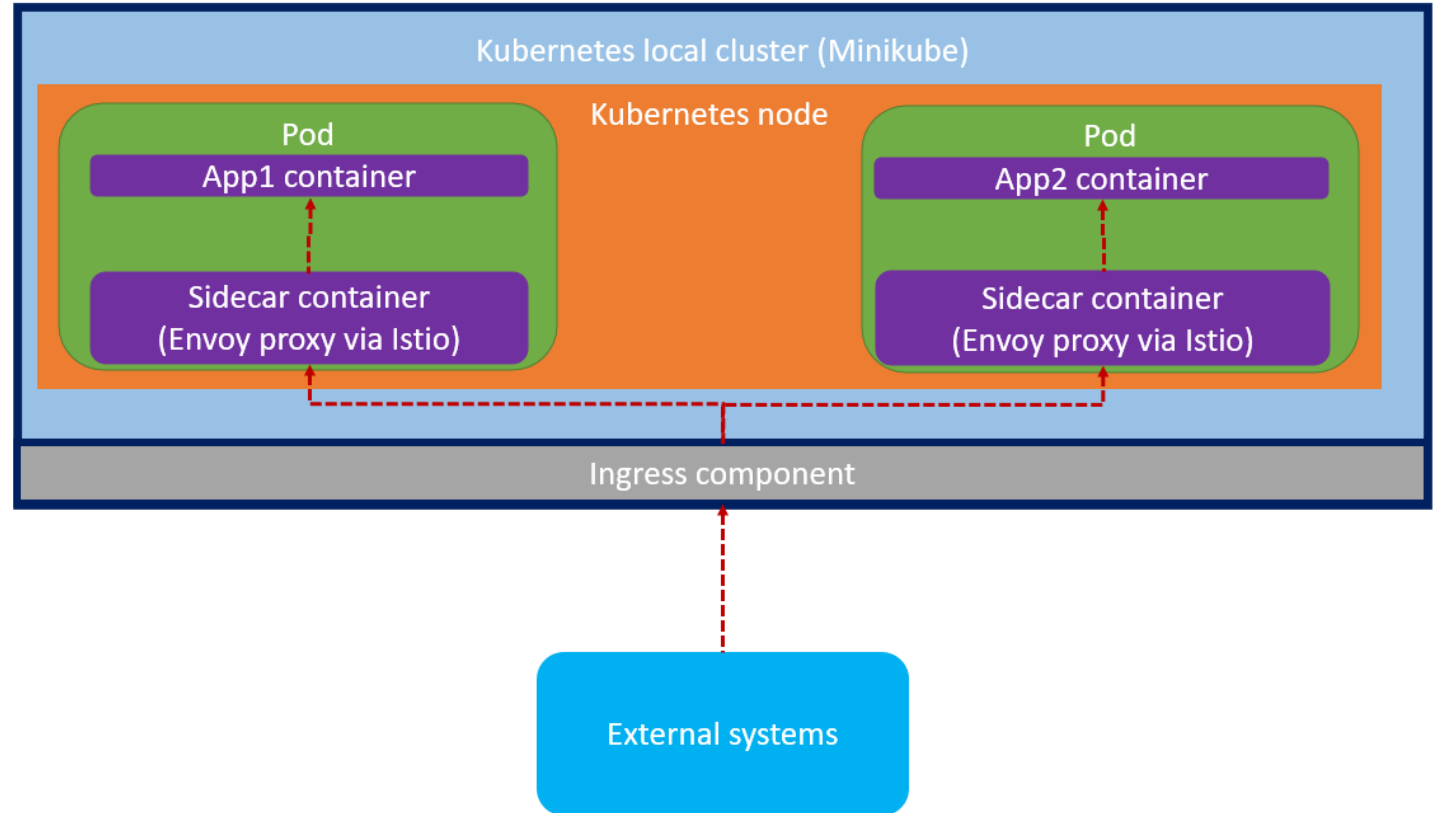


POC to validate my proposal

-  Achieve the following security aspects, only using the Kubernetes or Istio security features:
 - Authentication via a JWT (JSON Web Token).
 - Authorization via the claims of the JWT.
-  Nothing implemented at application level!



POC to validate my proposal





POC to validate my proposal

Authentication rules

- For both apps, the JWT token provided must:
 - ✓ Been issued by “**excellium-ias**” issuer.
 - ✓ Been signed with the RSA private key associated with this **public key**.



POC to validate my proposal

Authorization rules

- For **App1**:
 - ✓ A valid JWT token must be provided.
 - ✓ The **audience** claim of the token must be intended for **app1**.
- For **App2**:
 - ✓ A valid JWT token must be provided.
 - ✓ The **audience** claim of the token must be intended for **app2**.
 - ✓ The custom claim named **ispartner** must be set to **Yes**.



POC to validate my proposal

```
→ poc ./poc-deploy.sh
```

1



POC to validate my proposal

```
$ curl -sI $URL | head -1
```

```
[+] Create POC namespace ... 2
namespace/my-poc created
namespace/my-poc labeled
[+] Create Deployment & Service for both app ...
deployment.apps/my-app1-deployment created
service/my-app1-service created
deployment.apps/my-app2-deployment created
service/my-app2-service created
ingress.networking.k8s.io/my-app-ingress created
[+] Let all pods be initialized ...
[+] Status - Pod/Deployments/Service ...
>>>> Pods
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
my-app1-deployment-688c947759-mjgkw 2/2     Running   0           17s   10.244.0.206   minikube   <none>           <none>
my-app2-deployment-69dfc4bd8c-tl5tx 2/2     Running   0           17s   10.244.0.205   minikube   <none>           <none>
>>>> Deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES                               SELECTOR
my-app1-deployment 1/1     1             1           19s   my-app1      kicbase/echo-server:latest         app=my-app1
my-app2-deployment 1/1     1             1           19s   my-app2      kicbase/echo-server:latest         app=my-app2
>>>> Services
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE   SELECTOR
my-app1-service     NodePort    10.102.71.117 <none>        8080:31571/TCP   21s   app=my-app1
my-app2-service     NodePort    10.103.19.248 <none>        8080:30674/TCP   21s   app=my-app2
[+] Get Ingress IP address & test access to both app before adding authentication/authorization rules ...
Ingress IP is 192.168.49.2
>>>> http://192.168.49.2/app1
HTTP/1.1 200 OK
>>>> http://192.168.49.2/app2
HTTP/1.1 200 OK
[+] Add authentication/authorization rules ...
```

✗ At this stage, both instances of the apps are deployed **but without any authentication and authorization rules...**



POC to validate my proposal

```
[+] Add authentication/authorization rules ...
requestauthentication.security.istio.io/my-app1-authenticationrule created
requestauthentication.security.istio.io/my-app2-authenticationrule created
authorizationpolicy.security.istio.io/my-app1-authorizationrule created
authorizationpolicy.security.istio.io/my-app2-authorizationrule created
[+] Let all policies be applied ...
[+] ReTest access to both app ...
Ingress IP is 192.168.49.2
>>>> http://192.168.49.2/app1
HTTP/1.1 403 Forbidden
>>>> http://192.168.49.2/app2
HTTP/1.1 403 Forbidden
```

🤔 At this stage, authentication and authorization rules were applied and seem effective...

🔬 I need to validate that they are really effective!



POC to validate my proposal

```
→ poc python poc-test.py  
[+] Load signing key from JWKS
```

POC to validate my proposal

```
→ poc python poc-test.py
[+] Load signing key from JWKS
Loaded.
---
Tests for the App1
[+] Call app1 without a JWT token
HTTP 403 ⇒ RBAC: access denied
[+] Call app1 with a JWT token having the wrong issuer but the correct audience and signer
HTTP 401 ⇒ Jwt issuer is not configured
[+] Call app1 with a JWT token having the correct issuer and correct signer but a wrong audience (audience set to app2)
HTTP 403 ⇒ RBAC: access denied
[+] Call app1 with a JWT token having the correct issuer, correct signer and correct audience
HTTP 200 ⇒ Request served by my-app1-deployment-688c947759-mjgkw
```

✓ Authentication and authorization rules are effective for the app1.



POC to validate my proposal

Tests for the App2

```
---  
[+] Call app2 without a JWT token  
HTTP 403 ⇒ RBAC: access denied  
[+] Call app2 with a JWT token having the correct issuer, correct signer, correct audience but without the ispartner claim  
HTTP 403 ⇒ RBAC: access denied  
[+] Call app2 with a JWT token having the correct issuer, correct signer, correct audience but with the ispartner claim having a wrong value  
HTTP 403 ⇒ RBAC: access denied  
[+] Call app2 with a JWT token having the correct issuer, correct signer, correct audience but with the ispartner claim having a correct value using a wrong case  
HTTP 403 ⇒ RBAC: access denied  
[+] Call app2 with a JWT token having the correct issuer, correct signer, correct audience and correct ispartner claim value  
HTTP 200 ⇒ Request served by my-app2-deployment-69dfc4bd8c-tl5tx  
---
```

✓ Authentication and authorization rules are effective for the app2 too.



Conclusion

- Cloud Native Applications change the core structure of what is an “application” **by exploding it in several parts.**
- Each of them having its own lifecycle, technology stack, team, and security maturity.



Conclusion

- It is important to leverage the new security features provided by the common layer.
- Make the security level consistent across all parts, and do it in the most transparent way for Dev and Ops teams.



 All sources used are mentioned on additional slides.

EXCELLIUM
Cyber Solutions by Thales

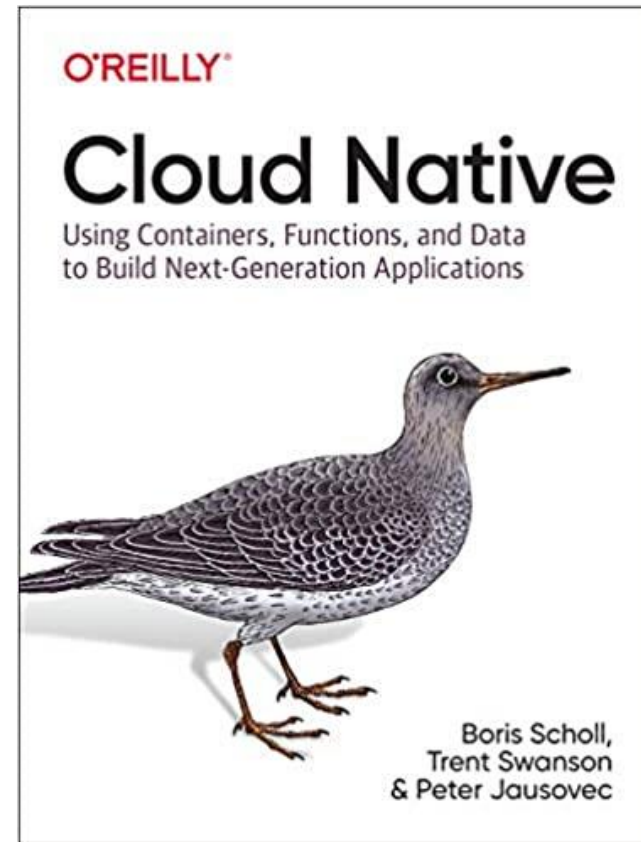
Questions & answers

Don't hesitate if you have some questions



We are open to any suggestions.

Sources & references



https://www.amazon.fr/Cloud-Native-Containers-Next-generation-Applications/dp/1492053821/ref=sr_1_1

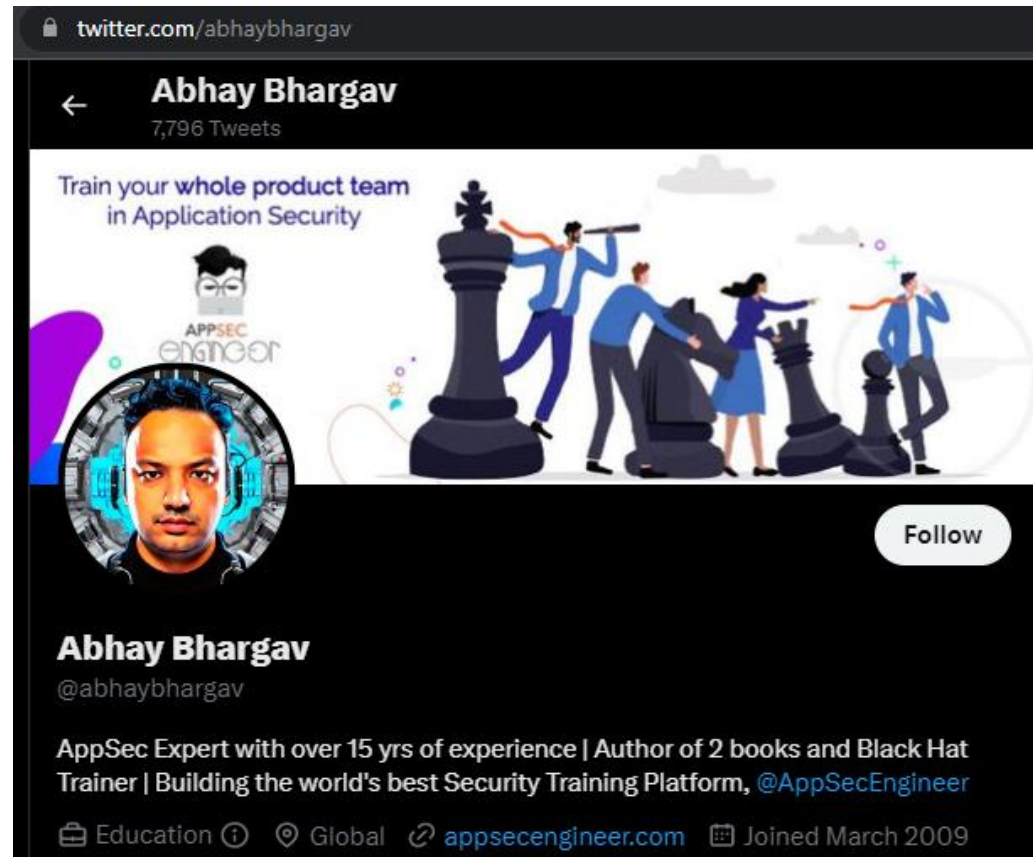
Sources & references



<https://www.amazon.fr/Understanding-Kubernetes-visual-way-sketchnotes/dp/B0BB619188/>

<https://twitter.com/aurelievache>

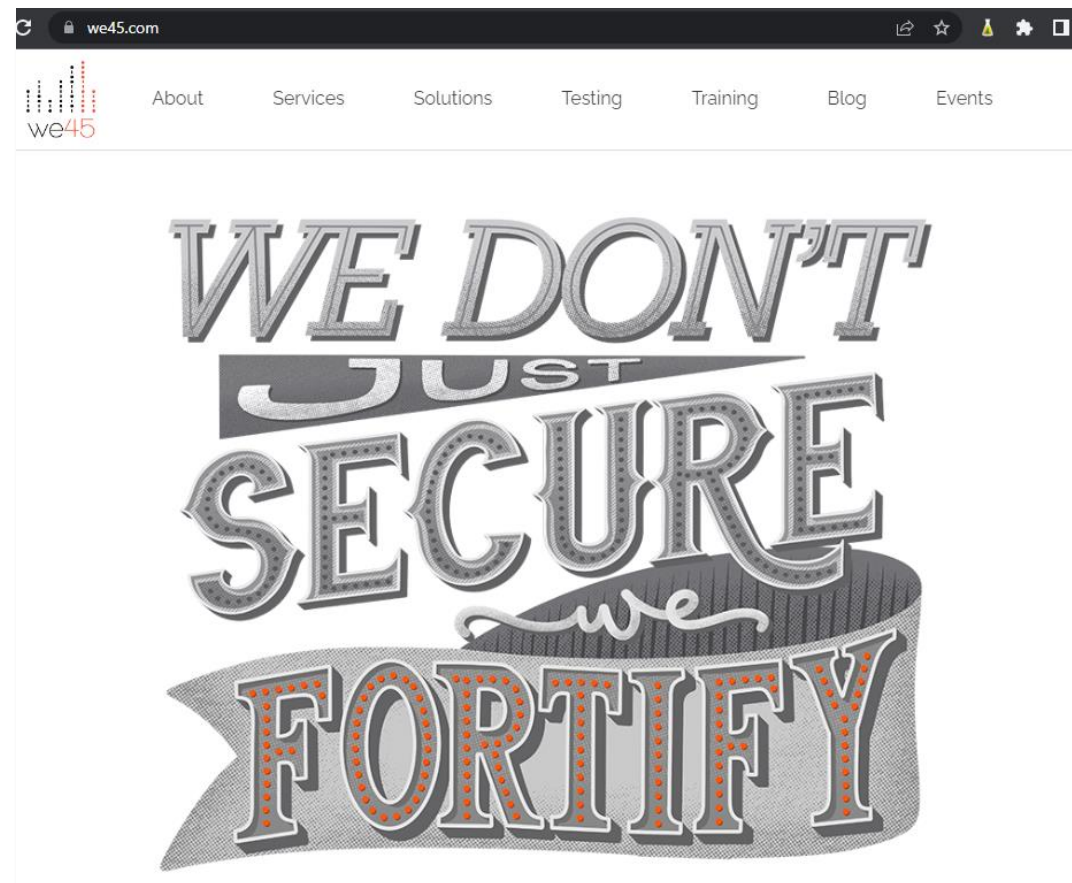
Sources & references



<https://twitter.com/abhaybhargav>

 <https://twitter.com/abhaybhargav/status/1662146295962673153>

Sources & references



<https://www.we45.com/> - <https://www.appsecengineer.com/>



- Why Kubernetes native instead of cloud native?
- Serverless Containers in Kubernetes environments



Cyber Solutions by Thales

Excellium Services S.A.
5 rue Goell
L-5326 Contern

Excellium Services Belgium N.V.
Orion Bldg, Belgicastraat 13
B-1930 Zaventem, Belgium

www.excellium-services.com
<https://ccp.excellium-services.com>